

JavaScript Async Callbacks

[Back to JS page](#)

Note: This topic builds on the Callbacks fundamentals covered in [05.2012-js-Callbacks](#).

Table of Contents

- [JavaScript Async Callbacks](#)
 - [Synchronous Callbacks](#)
 - [Example 1](#)
 - [Asynchronous Callbacks](#)
 - [Example 2](#)
 - [Callback Chains](#)
 - [Example 3](#)
 - [Document](#)
 - [Reference](#)

Synchronous Callbacks

A synchronous callback is executed during the execution of the higher-order function. Array methods like `forEach()` use synchronous callbacks:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Synchronous Callbacks</h4>
<p id="demo"></p>

<script>
const numbers = [1, 2, 3, 4, 5];
let text = "";

// forEach uses a synchronous callback
numbers.forEach(function(value) {
  text += value * 2 + " ";
});

document.getElementById("demo").innerHTML = "Doubled values: " + text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Asynchronous Callbacks

An asynchronous callback is executed after the higher-order function has completed.

`setTimeout()` is a classic example:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Asynchronous Callbacks</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myDisplayer(something) {
  document.getElementById("demo2").innerHTML = "Result: " + something;
}

function myCalculator(num1, num2, myCallback) {
  let sum = num1 + num2;
  myCallback(sum);
}

document.getElementById("demo1").innerHTML = "Calculating...";
myCalculator(5, 5, myDisplayer);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Callback Chains

When one callback depends on the result of another, you get callback chains (sometimes called "callback hell"):

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Callbacks</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Callback Chains</h4>
<p id="demo"></p>

<script>
function step1(callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 1: Data loaded<br>";
    callback("Data from step 1");
  }, 1000);
}

function step2(data, callback) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 2: Processing " + data + "<br>";
    callback("Processed data");
  }, 1000);
}

function step3(data) {
  setTimeout(function() {
    document.getElementById("demo").innerHTML += "Step 3: Displaying " + data + "<br>";
    document.getElementById("demo").innerHTML += "All steps complete!";
  }, 1000);
}

// Callback chain
step1(function(result1) {
  step2(result1, function(result2) {
    step3(result2);
  });
});
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Async Callbacks](#)